

MeroSwasthya Hospital Management System

Project report for a role-based healthcare web application that connects patients, doctors, hospitals, and administrators in one platform.

Report Date: May 12, 2026

Project Type: MERN-style web application

Prepared For: Project Manager Review

Status: Functional development build verified

Table of Contents

1. Executive Summary	9. Database Design
2. Objectives and Scope	10. API Overview
3. Technology Stack	11. Security Notes
4. Architecture Overview	12. Setup and Configuration
5. User Roles	13. Verification Status
6. Main Features	14. Future Work
7. Frontend Details	15. Conclusion
8. Backend Details	

1. Executive Summary

MeroSwasthya is a Hospital Management System built to make healthcare coordination easier for patients, doctors, hospitals, and administrators. Instead of keeping appointment booking, doctor availability, medical reports, and communication in separate places, the application brings these workflows into one web platform.

The project has a React frontend and a Node.js/Express backend connected to MongoDB. It includes role-based dashboards, OTP-based account verification, hospital and doctor management, queue-aware appointment booking, medical report generation, visit history, profile images, and real-time chat between patients and doctors.

In simple terms, the system helps patients find care, helps doctors manage appointments and reports, helps hospitals coordinate doctors and schedules, and helps admins keep the platform organized.

Key Points

- **Purpose:** Provide a single web-based workspace for common hospital and patient service workflows.
- **Main Users:** Patients, doctors, hospital users, and system administrators.
- **Core Benefit:** Reduces manual coordination by connecting bookings, doctor availability, chat, and reports.
- **Current State:** The frontend builds successfully and the major workflows are implemented in the codebase.
- **Next Priority:** Add automated testing, production security hardening, and deployment configuration before production use.

2. Objectives and Scope

The main objective of MeroSwasthya is to provide a practical healthcare management platform that can support everyday hospital operations while staying simple enough for patients to use.

Objectives

- Allow users to register, verify their account, and sign in securely.
- Give each user role a dashboard focused on their actual work.
- Allow hospitals to maintain profiles and manage doctor availability.
- Allow patients to search hospitals, choose doctors, and book appointments.
- Allow doctors and hospitals to create reports for completed or confirmed visits.
- Allow patients and doctors to communicate through real-time chat.
- Maintain visit history so patients can open reports from the correct appointment.

Current Scope

- Authentication, OTP verification, password reset, and profile image upload.
- Admin dashboard for users, hospitals, doctor approvals, and reports.
- Hospital dashboard for hospital profile, linked doctors, availability, appointments, and reports.
- Doctor dashboard for profile, appointment status, chat, and medical reports.
- Patient dashboard for profile, hospital search, doctor selection, booking, chat, visit history, and reports.
- Landing page with hospital image, sectioned content, testimonials, and call-to-action.

3. Technology Stack

Area	Technology	Use in Project
Frontend	React 18, Vite, React Router	Single page application, routing, dashboards, landing page, and build process.
Backend	Node.js, Express 5	REST API, route handling, middleware, and server setup.
Database	MongoDB, Mongoose	Schema-based document storage for users, appointments, hospitals, doctors, reports, and messages.
Authentication	JWT, bcryptjs	Token-based login and hashed password storage.
Email	Nodemailer	OTP verification and password reset email delivery.

Area	Technology	Use in Project
Realtime	Socket.IO	Live chat between patients and doctors.
Uploads	Multer	Profile image upload with type and size validation.

4. Architecture Overview

The project follows a client-server structure. The frontend runs as a React application and communicates with the backend using REST API requests. The backend is responsible for authentication, validation, database operations, role permissions, and realtime chat events.

MongoDB is used as the main database. Each major business object has its own Mongoose model. The backend keeps route definitions, controllers, services, middleware, and models separate, which makes the code easier to understand and extend.

Architecture Flow

1. A user opens the landing page and chooses Login or Get Started.
2. The user registers or logs in through the frontend.
3. The backend validates credentials and returns a JWT token after successful login or OTP verification.
4. The frontend stores the token and uses it for protected API requests.
5. The backend checks the token and role before allowing protected actions.
6. MongoDB stores account, profile, appointment, report, hospital, doctor, and chat data.
7. Socket.IO handles realtime chat using the same JWT identity.

5. User Roles

Role	Description	Main Responsibilities
Patient	Person looking for care and appointments.	Maintain profile, search hospitals, book doctors, chat, and view reports.
Doctor	Clinical user who handles appointments and reports.	Create profile, manage appointment status, chat with patients, and create reports.
Hospital	Hospital-side user managing hospital operations.	Maintain hospital profile, link doctors, set availability, view appointments, and create reports.
Admin	System-level user responsible for platform control.	Create users, create hospitals, approve doctors, view statistics, and review reports.

6. Main Features

Authentication

- Users can register with name, email, password, role, and optional profile image.
- OTP is sent by email for account verification.
- Users must verify OTP before login.
- JWT tokens are used to access protected routes.
- Password reset is available through email OTP.

Admin Features

- View system counts for users, patients, doctors, and hospitals.
- Create users for different roles.
- Create hospitals and connect them to hospital-admin users.
- Approve doctor profiles before doctors can receive bookings.
- Review reports across the system.

Hospital Features

- Create and update hospital profile details.
- Search registered doctors and link them to a hospital.
- Set doctor availability using dates and patient limits.
- View appointments booked under the hospital.
- Create medical reports for confirmed or completed appointments.

Doctor Features

- Create and update clinical profile details.
- Wait for admin approval before becoming bookable.
- View assigned appointments.
- Update appointment status such as confirmed, completed, cancelled, or no-show.
- Create and update patient medical reports.
- Reply to patient chats after a patient starts the conversation.

Patient Features

- Maintain personal health profile, blood group, allergies, and emergency contact.
- Search hospitals and view hospital cards with images.
- Open hospital details and see available doctors.

- Book appointments based on available dates.
- Track appointment queue/token numbers and status.
- Start chat with doctors.
- View visit history and open medical reports from the related visit.

7. Frontend Details

The frontend is designed as a role-aware single page application. It has public pages for landing, login, registration, and OTP verification. After login, the protected dashboard renders different content depending on the user's role.

Frontend Pages

- **Landing Page:** Public first page with hospital hero image, feature sections, testimonials, and Get Started button.
- **Login Page:** Allows verified users to sign in.
- **Register Page:** Allows new users to create an account.
- **Verify OTP Page:** Allows users to verify their email and resend OTP.
- **Dashboard Home:** Chooses the correct dashboard based on role.
- **Patient Dashboard:** Handles patient profile, hospital search, booking, chat, and history.
- **Doctor Dashboard:** Handles profile, appointments, chat, and reports.
- **Hospital Dashboard:** Handles hospital profile, doctors, appointments, and reports.
- **Admin Dashboard:** Handles stats, users, hospitals, approvals, and system reports.

UI Improvements Completed

- Added a clean landing page with hospital hero image.
- Divided landing page into clear sections.
- Added testimonials with profile images.
- Added clear Get Started CTA buttons.
- Removed raw JSON response panels from dashboards.
- Improved patient history so reports open from visit items.
- Added doctor and patient images in search and chat where available.
- Reduced hospital search card image size for cleaner browsing.

8. Backend Details

The backend is organized around route groups and controllers. Each route group handles one domain of the application. Middleware checks authentication and roles before protected controller logic runs.

Backend Structure

- `server.js` : Starts Express, Socket.IO, middleware, static uploads, and API routes.
- `db.js` : Connects the backend to MongoDB.
- `routes/` : Defines endpoint paths and attaches middleware.
- `controllers/` : Handles request logic and responses.
- `models/` : Defines Mongoose schemas.
- `middleware/` : Handles auth, role checks, and uploads.
- `services/` : Contains shared chat business logic.
- `sockets/` : Handles Socket.IO chat events.
- `utils/` : Contains email and dashboard helper logic.

9. Database Design

Model	Purpose	Important Fields
User	Stores login and role information.	name, email, password, role, isVerified, OTP fields, phone, profileImage.
PatientProfile	Stores patient health details.	user, dateOfBirth, gender, bloodGroup, address, emergency contact, allergies, chronic conditions.
Doctor	Stores doctor professional profile.	user, specialty, licenseNumber, experienceYears, consultationFee, maxDailyBookings, isApproved.
Hospital	Stores hospital information.	name, address, phone, email, bannerImage, adminUser, isActive.
DoctorHospital	Links doctors to hospitals.	doctor, hospital, availableDates, availabilitySlots, maxDailyBookings, isActive.
Appointment	Stores appointment and queue data.	patientUser, doctor, hospitalId, scheduledAt, queueDate, queueNumber, reason, status.
MedicalReport	Stores medical report for an appointment.	appointment, patientUser, doctor, hospitalId, diagnosis, prescription, tests, followUpDate, notes.

Model	Purpose	Important Fields
ChatMessage	Stores patient-doctor chat messages.	patientUser, doctor, senderUser, message, timestamps.

10. API Overview

Route Group	Purpose
/api/auth	Registration, login, OTP verification, password reset, current user, and profile image APIs.
/api/admin	System stats, user creation, and hospital creation by admin.
/api/doctor	Doctor listing, doctor profile, pending approvals, and doctor approval.
/api/hospital	Hospital listing, hospital profile, doctor candidates, doctor linking, and availability.
/api/patient	Patient profile and patient visit history.
/api/appointment	Appointment booking, appointment listing, and appointment status updates.
/api/report	Medical report creation, listing, and detail view.
/api/chat	Chat conversations, chat messages, and message sending.

11. Security Notes

Current Security Work

- Passwords are hashed using bcrypt.
- Protected endpoints require JWT authentication.
- Role middleware restricts access to role-specific routes.
- Socket.IO checks JWT before allowing chat events.
- Report access checks patient, doctor, hospital, or admin ownership.
- Profile image uploads are limited by file type and size.

Production Security Recommendations

- Restrict CORS to the real frontend domain.
- Hash OTP values before saving them in the database.
- Use cryptographically secure OTP generation.

- Add rate limiting for login, OTP, reset password, and chat actions.
- Add stricter request validation for all incoming payloads.
- Add audit logs for admin approval, report creation, and profile changes.

12. Setup and Configuration

Backend Setup

1. Open the `backend` directory.
2. Run `npm install`.
3. Create a `.env` file.
4. Run `npm run dev` for development.
5. Run `npm start` for production-style start.

Backend Environment Variables

- `MONGO_URI`: MongoDB connection string.
- `JWT_SECRET`: Secret key used for JWT signing.
- `EMAIL_USER`: Email account used to send OTP.
- `EMAIL_PASS`: Email account password or app password.
- `EMAIL_FROM`: Optional email sender address.
- `ADMIN_SECRET`: Secret used for admin registration.
- `PORT`: Optional backend port, default is 5000.

Frontend Setup

1. Open the `frontend` directory.
2. Run `npm install`.
3. Run `npm run dev` for development.
4. Run `npm run build` to create a production build.

The frontend API URL defaults to `http://localhost:5000/api`. It can be changed with `VITE_API_BASE_URL`.

13. Verification Status

- The frontend production build was tested with `npm run build`.
- The build completed successfully.

- Backend syntax checks were performed during development review for changed backend files.
- The backend test script is currently a placeholder and should be replaced with real tests.
- Full end-to-end testing requires a running MongoDB database and valid email credentials.

14. Future Work

The current system is a strong base for demonstration and continued development. Before using it in a production environment, the following work should be prioritized.

- **Add automated testing.** The project should include unit tests for service logic, controller tests for important workflows, and integration tests for login, role access, appointment booking, report creation, and chat. This will make future changes safer and help catch regressions early.
- **Improve production security.** CORS should be restricted, OTPs should be generated securely and stored as hashes, rate limiting should be added, and all request payloads should be validated. These changes are important because the system handles sensitive healthcare and user data.
- **Add deployment and environment documentation.** The project needs deployment instructions for frontend hosting, backend hosting, environment variables, MongoDB setup, file storage, and email configuration. This will make it easier for another team member or DevOps person to deploy the application reliably.
- **Enhance appointment and notification workflows.** Future versions can include appointment cancellation requests, reminders, email/SMS notifications, calendar views, and doctor schedule management. These improvements would make the platform more useful in real hospital operations.
- **Move uploads to cloud storage.** Profile images and future report attachments are currently stored locally on the backend server. For production, cloud or object storage should be used so files remain available across deployments, scaling, backups, and server restarts.

15. Conclusion

MeroSwasthya is a practical healthcare management project that brings several important hospital workflows into one place. The application is not just a basic login-and-dashboard system; it already includes meaningful role-based behavior for patients, doctors, hospitals, and administrators. Each role has a clear purpose inside the platform, which makes the system feel closer to a real-world hospital workflow than a simple demo application.

From the patient side, the system focuses on making care easier to access. Patients can create their profile, search hospitals, view hospital doctors, book an appointment, chat with a doctor, and later view reports from their visit history. This is useful because patients often lose track of where their appointments, reports, and doctor communication are stored. In this project, those details are connected through one account and one dashboard.

From the doctor side, the platform supports the daily work of handling appointments and medical documentation. A doctor can create a professional profile, wait for approval, view assigned appointments, update appointment status, chat with patients, and create medical reports. The approval step is important because it gives administrators control over which doctors become active in the system.

From the hospital side, the system provides tools to manage hospital information, connect doctors, set date-based availability, and view hospital appointments. The availability and queue logic is one of the stronger parts of the system because it helps avoid unlimited bookings and gives patients a queue/token number. This makes the booking flow more organized and closer to how many hospitals actually manage patient visits.

From the admin side, the project includes system-level control for creating users, creating hospitals, approving doctors, viewing reports, and checking basic statistics. This gives the application a management layer that can support future growth. As more users and hospitals are added, the admin role becomes important for keeping the platform clean and controlled.

Technically, the project is structured in a maintainable way. The backend separates routes, controllers, services, models, middleware, sockets, and utilities. The frontend separates pages, dashboards, common components, API helpers, context, and styling. This structure will help future developers understand where to add new features. The use of React, Express, MongoDB, JWT, Multer, Nodemailer, and Socket.IO is appropriate for the current scope.

The recent UI improvements also make the project easier to present. The landing page now gives the application a proper first impression, the dashboards are cleaner, raw JSON response panels have been removed, hospital and doctor images improve the visual experience, and chat now shows user images. These changes make the application feel more complete and more understandable for non-technical reviewers.

At the same time, the project should not be considered production-ready yet. Important production work remains, especially automated tests, stricter security, better validation, deployment configuration, cloud file storage, and operational monitoring. These are normal next steps for moving a functional development project toward a reliable production system.

Overall, MeroSwasthya is a strong foundation for a hospital management system. It demonstrates the main workflows clearly and gives enough structure for continued development. With testing, security improvements, and deployment preparation, it can grow into a more complete healthcare platform that supports real patient, doctor, hospital, and administrative operations.